

Day 4 - Mini Project Documentation

Introduction

This day focused on building a complete console-based Student Progress Tracker in C. The project was developed step by step by extending the same program with new features while keeping it structured and stable.

Task 1 - Multiple Records

Implemented storage of multiple students using an array with a fixed capacity. Added functionality to insert multiple students and display all stored records. Used a count variable to track the current number of students.

Task 2 - Enum and Validation

Introduced enum for student status (Not Started, In Progress, Completed). Improved input validation for menu choices and user inputs. Used switch statements to control the program flow more clearly.

Task 3 - Data Analysis

Added report functionality using loops and conditions. Calculated total students, completed students, average progress, maximum and minimum values. Classified performance based on the average value.

Task 4 - Pointer Usage

Implemented pointer-based update functionality. Allowed modification of student data (progress and status) using references. Ensured that changes affect the original data and not copies.

Task 5 - Search and Evaluation

Added search functionality by ID. Displayed specific student data when found. Implemented evaluation logic using combined conditions to provide feedback.

Task 6 - Sorting and Ranking

Implemented sorting of students based on progress using nested loops. Displayed ranking from highest to lowest progress. Ensured all existing features continue working after sorting.

Task 7 - Edit and Delete

Extended the program with full data management. Implemented deletion of a student by ID. Shifted array elements after deletion to avoid gaps. Updated the count variable correctly and maintained consistency.

Task 8 - Final Structure and Documentation

In the final stage of the project, the code was reorganized to make it cleaner, more readable, and more stable. The program was divided into clear functions so that each part has a specific role. Repeated code was reduced by reusing helper functions, especially for input handling and student display. The console output and menu were also improved to make the information easier to read. In addition, a short project documentation was prepared to explain the program, the implemented features, the concepts used during the week, and the main difficulties faced during implementation.

Concepts Used

The project uses arrays to store multiple records, enums to represent status values, pointers to modify data directly, loops for processing and analyzing data, and switch statements for menu control.

Challenges Faced

Handling invalid input required careful validation. Using pointers correctly for updating data was challenging at first. Deleting elements and shifting the array without breaking the logic also required attention.

Final Improvements

In the final version, the code was cleaned and organized into clear functions. Repeated logic was reduced by using reusable functions. The console output and menu were improved to make the program more readable and user-friendly.

Conclusion

The project evolved into a complete and structured system. It supports adding, updating, searching, analyzing, sorting, and deleting student data. All features are integrated into a single menu-driven application.